Corso di laurea in Informatica (L-31)

Algoritmi e Strutture Dati

Algoritmi di ordinamento

Dai cicli alla ricorsione, dalle strutture dati ai limiti del confronto

---

Prof. Simone Faro

Indice

1 Il problema: perché mettere ordine? 2
1.1 Definizione formale . . . 2
1.2 Quali proprietà ci interessano? . . . 2
1.3 Il modello di calcolo e le convenzioni . . . 3

2 Tre strategie iterative: scegliere, inserire, scambiare 3
2.1 Selection Sort: scegliere il prossimo elemento . . . 3
2.2 Insertion Sort: far crescere un ordine già esistente . . . 4
2.3 Bubble Sort: l'ordine emerge da confronti locali . . . 5
2.4 Tre invarianti, tre modi di pensare . . . 6

3 Cambiare linguaggio: dall'iterazione alla ricorsione 6
3.1 Selection Sort ricorsivo: togliere il minimo e ricominciare . . . 6
3.2 Insertion Sort ricorsivo: ordinare prima, inserire poi . . . 7
3.3 Divide et impera: dividere non basta, occorre farlo bene . . . 8
3.4 Merge Sort: prima si ordina, poi si fonde . . . 8
3.5 Quicksort: prima si separa, poi si ordina . . . 9
3.6 Una sintesi prima del prossimo salto . . . 11

4 Heapsort: quando una struttura dati cambia il gioco 11
4.1 Un albero nascosto dentro un array . . . 11
4.2 Heapify: ripristinare la proprietà di max-heap . . . 12
4.3 Costruire uno heap in tempo lineare . . . 13
4.4 Heapsort: Selection Sort con un alleato . . . 13

5 Oltre i classici: cinque idee da esplorare 15
5.1 Shellsort: inserire senza essere prigionieri dell'adiacenza . . . 15
5.2 Introsort: accorgersi quando una buona strategia sta fallendo . . . 16
5.3 QuickHeapsort: dalla partizione allo heap . . . 17
5.4 Fast-Insertion-Sort: inserire blocchi anziché elementi . . . 18
5.5 QuickMergesort: il buffer era già nell'array . . . 19

6 Possiamo fare meglio di $n \log n$ usando soltanto confronti? 20
6.1 Ogni confronto è una domanda con due risposte . . . 20
6.2 La dimostrazione del limite inferiore . . . 20
6.3 Che cosa significa "ottimo"? . . . 21

7 Oltre i confronti: usare la struttura delle chiavi 21
7.1 Counting Sort: contare invece di confrontare . . . 21
7.2 Radix Sort: ordinare una cifra alla volta . . . 23
7.3 Perché non abbiamo violato il limite inferiore . . . 25

8 Confronto finale e criteri di scelta 25
8.1 Un'ultima prospettiva . . . 26

1 Il problema: perché mettere ordine?

Un elenco di nomi, le transazioni di una banca, le misure registrate da un sensore, gli appuntamenti di un calendario: sono oggetti molto diversi, ma spesso diventano più facili da gestire quando sono disposti in un ordine preciso. Se una rubrica telefonica fosse mescolata a ogni consultazione, trovare un contatto richiederebbe in generale di sfogliarla quasi tutta. Se è ordinata alfabeticamente, possiamo invece scartare ripetutamente metà delle voci. L'ordinamento, dunque, non è soltanto un'operazione di preparazione dei dati: è un modo per rendere possibili altre operazioni più efficienti.

Non sorprende che gli algoritmi di ordinamento abbiano accompagnato l'informatica sin dalle sue origini. La loro forza didattica, tuttavia, è ancora più importante della loro diffusione: pochi problemi permettono di osservare con tanta chiarezza la differenza fra un'idea semplice e un'idea efficiente, fra iterazione e ricorsione, fra un algoritmo che lavora direttamente sui dati e uno che si appoggia a una struttura dati costruita per aiutarlo.

1.1 Definizione formale

Definizione 1.1 (Problema dell'ordinamento)**.** Dato un insieme di n record $A[0], A[1], \ldots, A[n-1]$ e una relazione di ordine totale $\leq$ definita sulle loro *chiavi*, trovare una permutazione dei record tale che, nell'array risultante,

$$\text{chiave}(A[0]) \leq \text{chiave}(A[1]) \leq \cdots \leq \text{chiave}(A[n-1]).$$

Ogni record dell'input deve comparire esattamente una volta nell'output.

Per esempio, l'array

$$[\,7,\ 2,\ 5,\ 2,\ 9,\ 1\,] \qquad \longrightarrow \qquad [\,1,\ 2,\ 2,\ 5,\ 7,\ 9\,]$$

è stato ordinato in senso non decrescente. La richiesta di conservare esattamente gli elementi è essenziale: un procedimento che sostituisse ogni voce con il numero 0 produrrebbe una sequenza ordinata, ma certamente non risolverebbe il problema.

Nella pratica i dati sono spesso *record*: uno studente è descritto, ad esempio, da una matricola, da un cognome e da una media. Ordinare per media significa confrontare una componente del record, ma spostare l'intero record. In seguito distingueremo il valore della chiave dai dati associati.

1.2 Quali proprietà ci interessano?

La *correttezza* di un algoritmo di ordinamento richiede due fatti: l'output deve risultare ordinato e deve essere una permutazione dell'input. La *complessità temporale* misura il numero di operazioni eseguite in funzione di n; in particolare, confronteremo numero di confronti, scambi e spostamenti. La *complessità spaziale ausiliaria* riguarda invece la memoria aggiuntiva rispetto alla memoria già occupata dall'array in ingresso.

Diremo che un ordinamento è *stabile* se, fra record con la stessa chiave, conserva il loro ordine relativo iniziale. Si pensi a tre pratiche arrivate nell'ordine (Bianchi, 8), (Rossi, 8), (Verdi, 6), in cui il numero indica la priorità. Ordinando per priorità in modo stabile, Bianchi dovrà continuare a precedere Rossi. La stabilità è importante quando si effettuano ordinamenti successivi per più criteri: l'ultimo ordinamento non deve distruggere l'ordine che era stato costruito fra chiavi uguali.

Un algoritmo è detto *in-place in senso stretto* se modifica l'array usando $O(1)$ celle ausiliarie; lo spazio occupato dalle chiamate ricorsive va contato. È invece *adattivo* quando trae beneficio dall'ordine

già presente nei dati. Queste qualità sono indipendenti: si può essere veloci ma non stabili, stabili ma dispendiosi in memoria, o efficienti soltanto su input già quasi ordinati.

1.3 Il modello di calcolo e le convenzioni

Partiremo dal *modello a confronti*: possiamo accedere agli elementi dell'array, confrontare due chiavi e scambiare o spostare record, senza sfruttare ulteriori informazioni sulla rappresentazione numerica delle chiavi. Useremo la notazione asintotica O, Ω e Θ; per esempio $T(n) = \Theta(n^2)$ significa che il numero di operazioni cresce come un multiplo di n^2, a meno di fattori costanti.

Per tutta la dispensa gli array sono indicizzati da 0 a $n-1$. Gli intervalli $A[\ell..r]$ comprendono *entrambi* gli estremi; un intervallo con $\ell > r$ è vuoto. Nel codice, scambia$(A[i], A[j])$ scambia due record completi, mentre casuale(ℓ, r) sceglie uniformemente un intero nell'intervallo specificato. Le operazioni su chiavi e indici di dimensione macchina costano tempo costante. Per brevità, nelle procedure scriveremo $A[i] < A[j]$ per indicare il confronto fra le chiavi dei due record, e similmente per gli altri operatori relazionali. Tutti gli algoritmi di ordinamento saranno invocati mediante una procedura principale con i soli argomenti A e n, rispettivamente l'array da ordinare e la sua lunghezza. Le procedure ausiliarie, che operano su intervalli o su strutture temporanee, possono richiedere parametri ulteriori. Quando passiamo un intervallo $A[p..q]$ a una chiamata ricorsiva, intendiamo una *vista* delle celle originarie, non una loro copia: accedere alla vista e modificarla non comporta duplicazione dell'array. Per un approfondimento generale sugli algoritmi di ordinamento, si vedano i testi classici di Knuth e Cormen e collaboratori [3, 10].

2 Tre strategie iterative: scegliere, inserire, scambiare

Il nostro primo obiettivo non è scoprire subito l'algoritmo più rapido, ma capire come costruire un programma corretto e giustificare il suo costo. Considereremo tre idee, tutte fondate su cicli: selezionare il minimo, inserire un elemento al posto giusto, scambiare elementi adiacenti. Hanno in comune il modello a confronti e l'assenza di strutture dati specializzate; differiscono però in ciò che mantengono ordinato durante il lavoro.

2.1 Selection Sort: scegliere il prossimo elemento

Immaginiamo di dover mettere in fila alcuni fogli numerati. Una strategia naturale consiste nel cercare il foglio più piccolo, collocarlo per primo, poi cercare il più piccolo fra quelli rimasti, e così via. È precisamente l'idea di *Selection Sort*.

Al passo i la porzione $A[0..i-1]$ contiene già i i elementi minori, nell'ordine corretto. Cerchiamo quindi il minimo del suffisso $A[i..n-1]$ e lo scambiamo con $A[i]$. La porzione definitivamente ordinata cresce di una posizione: non dovremo più toccarla.

Seguiamo l'algoritmo sull'input $[5, 2, 4, 1, 3]$. Al primo passo viene selezionato 1 e otteniamo $[1, 2, 4, 5, 3]$; al secondo il minimo del suffisso è già 2; al terzo 3 viene spostato davanti a 4; infine 4 precede 5. Dopo ogni passaggio il prefisso ottenuto è *definitivo*: gli elementi che vi si trovano non devono più essere modificati.

Perché funziona. La proprietà da mantenere, detta *invariante del ciclo*, è questa: prima del passo i, $A[0..i-1]$ contiene i i elementi più piccoli, ordinati, e tutti gli altri sono nel suffisso. È vera inizialmente perché il prefisso è vuoto. Il passo i sceglie il minimo del suffisso e lo colloca nella posizione i, preservando la proprietà. Quando rimane un solo elemento, anche quest'ultimo deve trovarsi nella posizione corretta.

Algoritmo 1 Selection Sort iterativo.

```
1:  procedura SelectionSort(A, n)
2:      per i ← 0 fino a n − 2 fai
3:          m ← i
4:          per j ← i + 1 fino a n − 1 fai
5:              se A[j] < A[m] allora
6:                  m ← j
7:              fine se
8:          fine per
9:          se m ≠ i allora
10:             scambia(A[i], A[m])
11:         fine se
12:     fine per
13: fine procedura
```

Quanto costa. Al primo passo eseguiamo $n-1$ confronti, al secondo $n-2$, e così via, fino a uno. Il numero totale è quindi

$$C(n) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = \Theta(n^2).$$

Questo risultato non dipende dall'ordine iniziale: anche un array già ordinato richiede la stessa scansione di tutti i suffissi. Gli scambi, invece, sono al più $n-1$ e lo spazio ausiliario è $O(1)$. Selection Sort è di regola *non stabile*: selezionando il minimo e scambiandolo con un elemento lontano, due record di pari chiave possono invertire il loro ordine relativo. Sull'input $[(2,a),(2,b),(1,c)]$ il primo scambio produce $[(1,c),(2,b),(2,a)]$.

Questo algoritmo insegna una prima lezione: ridurre gli scambi non significa necessariamente ridurre i confronti. Più avanti riprenderemo esattamente l'idea di "scegliere ogni volta il prossimo massimo", ma la renderemo molto più efficiente grazie a una struttura dati.

2.2 Insertion Sort: far crescere un ordine già esistente

Chi ha mai ordinato le carte da gioco in mano conosce un'altra strategia: prende una carta nuova e la inserisce nel punto corretto delle carte già ordinate. *Insertion Sort* fa lo stesso: mantiene un prefisso ordinato e, a ogni passo, vi inserisce il primo elemento che si trova ancora fuori dal prefisso.

Algoritmo 2 Insertion Sort iterativo.

```
1:  procedura InsertionSort(A, n)
2:      per i ← 1 fino a n − 1 fai
3:          x ← A[i]
4:          j ← i − 1
5:          mentre j ≥ 0 e A[j] > x fai
6:              A[j + 1] ← A[j]
7:              j ← j − 1
8:          fine mentre
9:          A[j + 1] ← x
10:     fine per
11: fine procedura
```

Prendiamo $[3,1,4,2]$. Dopo l'inserimento di 1 abbiamo $[1,3,4,2]$. Il 4 è già al suo posto e non

richiede spostamenti. Infine, il 2 arretra idealmente tra 1 e 3, mentre 3 e 4 scorrono a destra: il risultato è $[1, 2, 3, 4]$.

Perché funziona. All'inizio del passo i, il prefisso $A[0..i-1]$ è ordinato. Durante il ciclo interno, gli elementi maggiori di x si spostano di una posizione verso destra, lasciando libero il posto in cui inserire x. Gli altri elementi conservano il loro ordine. Pertanto il nuovo prefisso $A[0..i]$ è ordinato: l'invariante è ristabilita.

Quanto costa. Nel caso migliore l'array è già ordinato: il confronto $A[i-1] > x$ fallisce immediatamente a ogni passo, per un totale di $\Theta(n)$ operazioni. Nel caso peggiore gli elementi arrivano in ordine decrescente: il passo i deve far scorrere i elementi, e

$$T(n) = \Theta\left(\sum_{i=1}^{n-1} i\right) = \Theta(n^2).$$

Più precisamente, il numero totale di spostamenti nel ciclo interno coincide con il numero di *inversioni* dell'input, cioè di coppie di posizioni $p < q$ per cui $A[p] > A[q]$. Ne segue un costo $O(n + I)$, dove I è il numero di inversioni: è la ragione matematica per cui Insertion Sort è adattivo.

Poiché spostiamo soltanto elementi *strettamente* maggiori di x, l'algoritmo è stabile. Utilizza un solo record temporaneo e pochi indici, quindi è in-place. È un esempio memorabile di compromesso: molto semplice, eccellente su input piccoli o quasi ordinati, ma quadratico nel caso peggiore.

2.3 Bubble Sort: l'ordine emerge da confronti locali

Una terza possibilità consiste nel confrontare soltanto coppie adiacenti. Se la coppia è nell'ordine sbagliato, la scambiamo. Scorrendo una volta l'intero array da sinistra a destra, il massimo dell'array arriva all'estremità destra, quasi fosse una bolla che risale. Ripetendo il passaggio sulla parte rimasta, otteniamo *Bubble Sort*.

Algoritmo 3 Bubble Sort con arresto anticipato.

```
1:  procedura BubbleSort(A, n)
2:      per t ← 0 fino a n − 2 fai
3:          cambiato ← falso
4:          per j ← 0 fino a n − t − 2 fai
5:              se A[j] > A[j + 1] allora
6:                  scambia(A[j], A[j + 1])
7:                  cambiato ← vero
8:              fine se
9:          fine per
10:         se cambiato = falso allora
11:             interrompi
12:         fine se
13:     fine per
14: fine procedura
```

L'invariante riguarda ora il *suffisso*: dopo t passaggi completi, gli ultimi t elementi sono i più grandi e si trovano nella posizione definitiva. Se durante una scansione non avviene alcuno scambio, tutte le coppie adiacenti risultano già ordinate; quindi lo è anche l'intero array e possiamo fermarci.

Senza arresto anticipato i confronti sarebbero sempre $n(n-1)/2$; con la variante indicata, il caso migliore scende a $\Theta(n)$, mentre il caso peggiore resta $\Theta(n^2)$. La memoria ausiliaria è costante. Inoltre, non scambiare mai record con chiavi uguali preserva la stabilità. Bubble Sort non è normalmente la soluzione più competitiva, ma aiuta a capire come un'informazione globale possa emergere ripetendo operazioni puramente locali.

2.4 Tre invarianti, tre modi di pensare

Selection Sort sa che il prefisso contiene *gli elementi definitivamente più piccoli*; Insertion Sort sa che il prefisso è *ordinato, ma non ancora definitivo*; Bubble Sort sa che il suffisso contiene *gli elementi definitivamente più grandi*. La differenza è sottile, ma decisiva: un algoritmo non è definito soltanto dalle operazioni che esegue, bensì dalla proprietà che riesce a conservare a ogni passo.

Tabella 1: Confronto fra i primi algoritmi. Le complessità temporali si riferiscono alla variante di codice descritta.

Algoritmo	Caso migliore	Caso peggiore	Spazio extra	Stabile
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	No
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	Sì
Bubble Sort ottimizzato	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	Sì

Il prossimo passo è cambiare il *modo di esprimere* queste stesse idee. Scopriremo che la ricorsione può semplicemente riscrivere un algoritmo iterativo, ma può anche permettere una scomposizione del problema che cambia radicalmente la complessità.

3 Cambiare linguaggio: dall'iterazione alla ricorsione

Un ciclo ripete un'azione finché resta del lavoro da fare. La ricorsione racconta lo stesso processo in modo differente: per risolvere un problema, risolviamo una sua versione più piccola e utilizziamo il risultato. Alla base c'è un *caso terminale*, che non richiede ulteriori chiamate; al passo ricorsivo c'è una riduzione effettiva della dimensione, che garantisce la terminazione.

È importante non attribuire alla ricorsione un potere che non possiede. Riscrivere un algoritmo con chiamate ricorsive non lo rende, di per sé, più rapido. Il cambiamento interessante si verifica quando la nuova formulazione suggerisce un diverso modo di suddividere il lavoro.

3.1 Selection Sort ricorsivo: togliere il minimo e ricominciare

Selection Sort contiene già una struttura ricorsiva nascosta. Per ordinare $A[0..n-1]$, scegliamo il minimo e mettiamolo in posizione 0. A quel punto rimane esattamente lo stesso problema, ma sulla vista del suffisso $A[1..n-1]$, che contiene un elemento in meno. Non occorre passare l'indice della prima cella: ogni chiamata lavora sul proprio array (o sulla propria vista) e sulla relativa lunghezza.

La chiamata iniziale è SELECTIONSORTRIC(A, n). Il caso base è immediato: un array di zero o un elemento è già ordinato. Nel caso ricorsivo collochiamo il minimo al primo posto e ordiniamo per ipotesi ricorsiva il suffisso rimanente. La vista $A[1..n-1]$ continua a riferirsi all'array originale e non crea copie dei record: così gli scambi effettuati dalle chiamate successive agiscono proprio sulle celle che restano da ordinare. Otteniamo un array corretto perché nessun elemento del suffisso può precedere il minimo.

Se $T(n)$ indica il tempo impiegato su una porzione di n elementi, allora

$$T(n) = T(n-1) + \Theta(n), \qquad T(1) = \Theta(1),$$

Algoritmo 4 Selection Sort ricorsivo.

```
1:  procedura SelectionSortRic(A, n)
2:      se n ≤ 1 allora
3:          restituisci
4:      fine se
5:      m ← 0
6:      per j ← 1 fino a n − 1 fai
7:          se A[j] < A[m] allora
8:              m ← j
9:          fine se
10:     fine per
11:     se m ≠ 0 allora
12:         scambia(A[0], A[m])
13:     fine se
14:     SelectionSortRic(A[1..n − 1], n − 1)
15: fine procedura
```

perché trovare il minimo richiede una scansione lineare. Espandendo la ricorrenza otteniamo di nuovo $T(n) = \Theta(n^2)$. La profondità delle chiamate è però $\Theta(n)$, dunque lo stack occupa $\Theta(n)$ memoria ausiliaria: la versione ricorsiva, diversamente da quella iterativa, non è in-place in senso stretto.

3.2 Insertion Sort ricorsivo: ordinare prima, inserire poi

Con Insertion Sort l'ordine delle operazioni si ribalta. Per ordinare $A[0..n-1]$, prima ordiniamo ricorsivamente il prefisso $A[0..n-2]$, poi inseriamo $A[n-1]$ nel prefisso già ordinato. La ricorsione non è più alla fine dell'algoritmo, bensì precede l'operazione caratteristica.

Algoritmo 5 Insertion Sort ricorsivo.

```
1:  procedura InsertionSortRic(A, n)
2:      se n ≤ 1 allora
3:          restituisci
4:      fine se
5:      InsertionSortRic(A, n − 1)
6:      x ← A[n − 1]
7:      j ← n − 2
8:      mentre j ≥ 0 e A[j] > x fai
9:          A[j + 1] ← A[j]
10:         j ← j − 1
11:     fine mentre
12:     A[j + 1] ← x
13: fine procedura
```

La chiamata iniziale è InsertionSortRic(A, n). A differenza di Selection Sort, qui non cambiamo l'inizio dell'array: la chiamata ricorsiva usa lo stesso A ma una lunghezza inferiore. La correttezza segue dalla stessa idea usata per l'algoritmo iterativo: l'ipotesi ricorsiva ci consegna un prefisso ordinato, e l'inserimento mantiene l'ordine preservando tutte le occorrenze. Anche la stabilità è identica, perché restano fermi i record con chiave uguale a x.

Nel caso peggiore, inserire l'ultimo elemento costa $\Theta(n)$ e otteniamo $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. Su un array già ordinato, il singolo inserimento costa $\Theta(1)$, perciò $T(n) = T(n-1) + \Theta(1) = \Theta(n)$.

Anche qui la profondità ricorsiva è $\Theta(n)$.

Una distinzione importante. Selection Sort *fa qualcosa e poi chiama* il problema più piccolo; Insertion Sort *chiama il problema più piccolo e poi fa qualcosa*. Questo cambia il punto del programma in cui si lavora, ma non cambia la dimensione del sottoproblema: in entrambi i casi si elimina un solo elemento a ogni livello. Per accelerare davvero, proviamo invece a dimezzare il problema.

3.3 Divide et impera: dividere non basta, occorre farlo bene

Il paradigma *divide et impera* ha tre momenti: si divide un problema in sottoproblemi, si risolvono i sottoproblemi (spesso ricorsivamente), si combinano i risultati. Non c'è una regola che imponga di creare due sottoproblemi, ma dividerne uno in due parti di dimensioni simili è spesso vantaggioso.

Immaginiamo un array di n elementi. Se a ogni chiamata ne eliminiamo uno solo, abbiamo quasi n livelli di ricorsione. Se la dimensione si dimezza, bastano circa $\log_2 n$ livelli. La differenza fra una catena lunga e un albero basso è il cuore della prossima accelerazione.

3.4 Merge Sort: prima si ordina, poi si fonde

L'idea di *Merge Sort* è sorprendentemente semplice: dividiamo l'array in due metà, ordiniamo ciascuna metà, poi le *fondiamo* in un'unica sequenza ordinata. Perché la fusione sia facile, dobbiamo saper risolvere una questione più piccola: come unire due sequenze già ordinate?

Se le sequenze sono $[1, 4, 8]$ e $[2, 3, 9]$, basta osservare i primi elementi non ancora consumati. Tra 1 e 2 scegliamo 1; tra 4 e 2 scegliamo 2; poi 3, poi 4, e così via. Ogni scelta è definitiva: il più piccolo tra i due elementi di testa è certamente il più piccolo di tutti gli elementi rimasti.

Questo procedimento di *fusione*, o *merge*, richiede tempo lineare nella somma delle lunghezze delle sequenze. Ed è sufficiente per trasformare la ricorsione in un algoritmo efficiente.

Algoritmo 6 Fusione stabile di due intervalli contigui già ordinati.

```
1:  procedura MERGE(A, ℓ, m, r, B)
2:      i ← ℓ; j ← m + 1; k ← ℓ
3:      mentre i ≤ m e j ≤ r fai
4:          se A[i] ≤ A[j] allora
5:              B[k] ← A[i]; i ← i + 1
6:          altrimenti
7:              B[k] ← A[j]; j ← j + 1
8:          fine se
9:          k ← k + 1
10:     fine mentre
11:     mentre i ≤ m fai
12:         B[k] ← A[i]; i ← i + 1; k ← k + 1
13:     fine mentre
14:     mentre j ≤ r fai
15:         B[k] ← A[j]; j ← j + 1; k ← k + 1
16:     fine mentre
17:     per k ← ℓ fino a r fai
18:         A[k] ← B[k]
19:     fine per
20: fine procedura
```

Algoritmo 7 Merge Sort ricorsivo con interfaccia A, n.

```
1:  procedura MergeSort(A, n)
2:      se n ≤ 1 allora
3:          restituisci
4:      fine se
5:      m ← ⌊n/2⌋
6:      MergeSort(A[0..m − 1], m)
7:      MergeSort(A[m..n − 1], n − m)
8:      alloca B[0..n − 1]
9:      Merge(A, 0, m − 1, n − 1, B)
10: fine procedura
```

La chiamata iniziale è MergeSort(A, n). Anche in questo caso le chiamate ricorsive ricevono soltanto un array (o una sua vista) e la relativa lunghezza. Per mantenere lo pseudocodice essenziale, il buffer temporaneo B viene creato soltanto dopo il ritorno delle chiamate ricorsive: in questo modo non vi sono buffer di fusione simultaneamente attivi su più livelli e il picco di spazio ausiliario resta $O(n)$. In un'implementazione ottimizzata si può invece allocare un unico buffer all'inizio e riutilizzarlo in tutte le fusioni mediante una procedura ausiliaria con indici espliciti. La verifica della correttezza è ricorsiva: i due sottoproblemi producono due metà ordinate; l'operazione Merge, scegliendo sempre il minore tra i due primi elementi disponibili, genera la loro unione ordinata senza perdere record.

L'analisi che cambia prospettiva. Per semplicità supponiamo che n sia una potenza di due. La complessità soddisfa

$$T(n) = 2T(n/2) + \Theta(n), \qquad T(1) = \Theta(1).$$

A ciascun livello dell'albero di ricorsione, il lavoro complessivo di fusione è $\Theta(n)$: prima una fusione lunga n, al livello precedente due fusioni lunghe $n/2$, ancora prima quattro fusioni lunghe $n/4$, e così via. I livelli sono $\log_2 n$, dunque

$$T(n) = \Theta(n \log n).$$

La stessa conclusione vale per ogni n, anche quando le due parti differiscono di un elemento. Il risultato non dipende dall'ordine dell'input: Merge Sort mantiene $\Theta(n \log n)$ nel caso migliore, nel medio e nel peggiore con l'implementazione proposta. Il prezzo è un array ausiliario di n record; lo stack richiede soltanto $O(\log n)$ spazio aggiuntivo. Usando $A[i] \le A[j]$ nella fusione, in caso di uguaglianza prevale l'elemento della metà sinistra: l'algoritmo è stabile.

Un episodio dalle origini dell'informatica. Nel 1945 John von Neumann lavorò a una routine per *mescolare*, nel senso di fondere, due sequenze ordinate nell'ambito del progetto EDVAC. Il lavoro è considerato un antecedente diretto del Merge Sort moderno. Il manoscritto relativo alla routine di fusione è stato studiato da Donald Knuth e, più recentemente, dallo storico Mark Priestley [9, 12]. La storia è istruttiva: l'idea di comporre risultati ordinati nasce quasi insieme alla programmazione dei primi elaboratori, quando rendere automatiche perfino operazioni oggi familiari costituiva un autentico progresso.

3.5 Quicksort: prima si separa, poi si ordina

Merge Sort divide l'array in due metà *per posizione* e poi impiega tempo per ricombinarle. *Quicksort* percorre la strada opposta: prima separa gli elementi *per valore*, poi ordina separatamente le due parti. È sufficiente scegliere un elemento detto *pivot* e partizionare l'array in modo che tutti gli elementi alla sua

sinistra siano più piccoli del pivot e quelli a destra siano maggiori o uguali. Una volta collocato il pivot tra le due parti, la sua posizione finale è già nota.

Per esempio, scegliendo il pivot 5 nell'array $[8, 3, 1, 7, 5, 2]$, possiamo ottenere una partizione del tipo

$$[\,3,\ 1,\ 2\,] \quad | \quad [\,5\,] \quad | \quad [\,8,\ 7\,].$$

Non abbiamo ancora ordinato le due parti esterne. Tuttavia sappiamo che nessun elemento a sinistra può finire a destra del pivot e viceversa; possiamo quindi risolvere le due parti indipendentemente. Non è necessaria una fase di fusione.

Algoritmo 8 Partizione a due regioni, con pivot in posizione r.

```
1:  procedura PARTITION(A, ℓ, r)
2:      p ← A[r]
3:      i ← ℓ
4:      per j ← ℓ fino a r − 1 fai
5:          se A[j] < p allora
6:              scambia(A[i], A[j])
7:              i ← i + 1
8:          fine se
9:      fine per
10:     scambia(A[i], A[r])
11:     restituisci i
12: fine procedura
```

Algoritmo 9 Quicksort ricorsivo con scelta casuale del pivot.

```
1:  procedura QUICKSORT(A, n)
2:      se n ≤ 1 allora
3:          restituisci
4:      fine se
5:      q ← casuale(0, n − 1)
6:      scambia(A[q], A[n − 1])
7:      p ← PARTITION(A, 0, n − 1)
8:      QUICKSORT(A[0..p − 1], p)
9:      QUICKSORT(A[p + 1..n − 1], n − p − 1)
10: fine procedura
```

La chiamata iniziale è $\textsc{QuickSort}(A, n)$. Le due chiamate ricorsive operano su viste dell'array originario, senza copiarne gli elementi. Grazie a questa convenzione non occorre aggiungere gli estremi degli intervalli alla firma dell'algoritmo. La correttezza si comprende usando l'invariante della partizione: prima di esaminare $A[j]$, gli elementi in $A[\ell..i-1]$ sono inferiori al pivot, quelli in $A[i..j-1]$ non lo sono, e $A[j..r-1]$ è ancora da esaminare. Quando la scansione termina, lo scambio finale mette il pivot nella sua posizione definitiva. La correttezza ricorsiva delle due parti conclude l'argomento.

Quando è veloce, quando no. Partizionare n elementi costa $\Theta(n)$. Se ogni pivot divide l'array in due parti di dimensione comparabile, abbiamo

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

Se invece il pivot è sempre il minimo o il massimo della porzione considerata, abbiamo una divisione estrema:

$$T(n) = T(n-1) + \Theta(n) = \Theta(n^2).$$

Nella versione proposta la scelta casuale del pivot evita che un particolare ordinamento di chiavi *tutte distinte* determini sistematicamente un caso sfavorevole: il numero *atteso* di confronti è $\Theta(n \log n)$. Non si tratta però di una garanzia nel caso peggiore.

Una precisazione sui duplicati è importante: la procedura PARTITION manda tutte le chiavi uguali al pivot dalla stessa parte. Se l'input contiene soltanto valori uguali, ogni divisione è estrema e il tempo è quadratico, anche scegliendo un pivot casuale. Una soluzione pratica è la *partizione a tre vie*, che distingue chiavi minori, uguali e maggiori del pivot e non richiama ricorsivamente la regione centrale.

Quicksort non è generalmente stabile. Nella formulazione ricorsiva diretta, lo stack occupa $O(\log n)$ spazio atteso per chiavi distinte e $O(n)$ nel caso peggiore; una gestione attenta della ricorsione, che richiama soltanto il sottoproblema più piccolo e prosegue iterativamente sull'altro, può limitarne l'uso a $O(\log n)$ anche nel caso peggiore.

Il nome e il metodo sono legati a C. A. R. Hoare, che concepì l'idea alla fine degli anni Cinquanta e la pubblicò formalmente nel 1962 [8]. Quicksort è diventato un classico non perché abbia il miglior limite nel caso peggiore, ma perché in molte implementazioni reali unisce semplicità, buona località di memoria e prestazioni notevoli.

3.6 Una sintesi prima del prossimo salto

La sola ricorsione non ha migliorato né Selection Sort né Insertion Sort: entrambi continuano a risolvere un problema più piccolo di *una* unità. Merge Sort e Quicksort hanno invece introdotto nuove strategie di decomposizione. Il primo garantisce $\Theta(n \log n)$ in ogni caso pagando memoria aggiuntiva; il secondo è spesso estremamente veloce, ma una divisione sfavorevole può costargli quadraticamente.

C'è ancora un'altra possibilità: mantenere l'idea elementare di Selection Sort e migliorare la *ricerca* dell'elemento da selezionare. Per farlo non serve inventare un nuovo paradigma di scomposizione: serve introdurre una struttura dati.

4 Heapsort: quando una struttura dati cambia il gioco

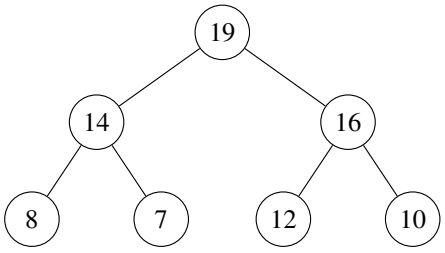
Ritorniamo per un momento a Selection Sort. L'algoritmo aveva un pregio: dopo aver scelto un elemento, sapeva esattamente dove metterlo e non aveva bisogno di ripensarci. Aveva però un difetto costoso: per individuare ogni volta il minimo doveva scandire quasi tutti gli elementi ancora disponibili. Possiamo conservare il primo pregio ed eliminare il secondo?

La risposta è sì, a patto di organizzare i dati in una struttura che permetta di individuare rapidamente il massimo e di ripristinare rapidamente la stessa proprietà dopo averlo rimosso. La struttura si chiama *heap binario*. Heapsort sarà il primo algoritmo della nostra trattazione a beneficiare non soltanto di cicli, confronti o chiamate ricorsive, ma di una rappresentazione dei dati progettata per rendere efficiente una precisa operazione.

4.1 Un albero nascosto dentro un array

Un *albero binario completo* ha tutti i livelli pieni, eccetto eventualmente l'ultimo, che viene riempito da sinistra verso destra. È possibile rappresentarlo senza puntatori, utilizzando direttamente un array: il nodo alla posizione i ha i figli alle posizioni $2i + 1$ e $2i + 2$, se esistono; il padre di un nodo con indice $i > 0$ è alla posizione $\lfloor (i-1)/2 \rfloor$.

Definizione 4.1 (Max-heap)**.** Un max-heap è un albero binario completo in cui la chiave di ogni nodo è maggiore o uguale alle chiavi dei suoi figli. Di conseguenza, il massimo si trova sempre alla radice, cioè nella cella $A[0]$.



i	0	1	2	3	4	5	6
$A[i]$	19	14	16	8	7	12	10

Figura 1: Lo stesso max-heap visto come albero e come array. Nessuna struttura con puntatori è necessaria.

Una proprietà cruciale è che *l'array di un heap non è interamente ordinato*. Per esempio, 14 e 16 possono apparire in qualunque ordine nelle celle dei due figli della radice. Abbiamo imposto soltanto i vincoli necessari a trovare subito il massimo: questo ordine *parziale* è molto meno costoso da mantenere di un ordinamento totale.

4.2 Heapify: ripristinare la proprietà di max-heap

Se sostituiamo la radice con un valore più piccolo, la proprietà di max-heap può essere violata soltanto lungo un cammino verso il basso. Confrontiamo il nuovo elemento con il maggiore dei due figli: se è già almeno altrettanto grande, ci fermiamo; altrimenti lo scambiamo con il figlio maggiore e proseguiamo. Questa operazione è chiamata *Heapify*: supponendo già corretti i sottoalberi dei figli, ripristina la proprietà di max-heap nel sottoalbero radicato nel nodo considerato.

Algoritmo 10 Ripristino della proprietà di max-heap nel prefisso $A[0..h-1]$.

```
1:  procedura HEAPIFY(A, i, h)
2:      mentre 2i + 1 < h fai
3:          c ← 2i + 1
4:          se c + 1 < h e A[c + 1] > A[c] allora
5:              c ← c + 1
6:          fine se
7:          se A[i] ≥ A[c] allora
8:              interrompi
9:          fine se
10:         scambia(A[i], A[c])
11:         i ← c
12:     fine mentre
13: fine procedura
```

Supponiamo che i sottoalberi dei figli di i siano già max-heap. Allora HEAPIFY produce un max-heap radicato in i: a ogni passo colloca nella posizione corrente il maggiore fra il valore corrente e i suoi figli, poi eventualmente propaga l'unica violazione restante nel sottoalbero scelto. Un heap con h nodi ha altezza $O(\log h)$, quindi una discesa costa $O(\log h)$ e usa $O(1)$ memoria aggiuntiva.

4.3 Costruire uno heap in tempo lineare

Potremmo inserire uno alla volta i valori in un heap inizialmente vuoto: avremmo un procedimento valido di costo $O(n \log n)$. Esiste però un modo più elegante. Le foglie sono già max-heap di un solo nodo. Partendo dall'ultimo nodo interno, cioè dall'indice $\lfloor n/2 \rfloor - 1$, e risalendo fino alla radice, invochiamo HEAPIFY su ogni nodo. Quando lavoriamo su un nodo, i suoi due sottoalberi sono già heap corretti.

Algoritmo 11 Costruzione dal basso di un max-heap (metodo bottom-up).

```
1: procedura BuildMaxHeap(A, n)
2:     per i ← ⌊n/2⌋ − 1 fino a 0, decrescendo fai
3:         Heapify(A, i, n)
4:     fine per
5: fine procedura
```

È naturale pensare che il costo sia $O(n \log n)$, perché effettuiamo $O(n)$ chiamate, ciascuna di costo al più $O(\log n)$. Questo è un limite superiore corretto, ma non è stretto: quasi tutti i nodi sono vicini alle foglie e richiedono discese molto brevi. Il numero di nodi con altezza h è $O(n/2^{h+1})$, quindi il costo totale è limitato da

$$O\left(n \sum_{h \geq 0} \frac{h}{2^h}\right) = O(n),$$

poiché la serie converge. D'altra parte occorre esaminare $\Omega(n)$ celle, dunque la costruzione è $\Theta(n)$.

4.4 Heapsort: Selection Sort con un alleato

Adesso l'algoritmo è quasi inevitabile. Costruiamo un max-heap. Il massimo si trova in $A[0]$; scambiamolo con l'ultima cella ancora appartenente allo heap. Il massimo è definitivamente collocato alla fine dell'array. Riduciamo di uno la dimensione dello heap e ripristiniamo la proprietà di max-heap nella parte rimasta. Ripetiamo fino a esaurimento.

Algoritmo 12 Heapsort in-place.

```
1: procedura HeapSort(A, n)
2:     BuildMaxHeap(A, n)
3:     per h ← n fino a 2, decrescendo fai
4:         scambia(A[0], A[h − 1])
5:         Heapify(A, 0, h − 1)
6:     fine per
7: fine procedura
```

L'invariante del ciclo è composta da due parti: il prefisso $A[0..h-1]$ è un max-heap; il suffisso $A[h..n-1]$ è già ordinato e contiene gli elementi più grandi dell'input, tutti maggiori o uguali agli elementi nel prefisso. Lo scambio trasferisce il massimo del prefisso alla sua posizione definitiva e HEAPIFY ricostruisce l'heap più piccolo. Al termine, l'intero array è ordinato.

Complessità. Costruire lo heap costa $\Theta(n)$; poi eseguiamo $n-1$ estrazioni, ciascuna con una discesa al più logaritmica. Il tempo complessivo, anche nel caso peggiore, è $O(n \log n)$, e nel caso peggiore questa stima è stretta: $\Theta(n \log n)$. Per l'implementazione descritta, input con tutte le chiavi uguali consentono invece discese immediate e un tempo $\Theta(n)$. La memoria aggiuntiva è $O(1)$, perché l'albero è codificato *dentro* l'array. Heapsort non è in generale stabile, per via degli scambi a distanza fra la radice e l'ultima posizione disponibile.

Il confronto con Selection Sort. I due algoritmi condividono la medesima decisione strategica: a ogni passo scegliere un elemento estremo e collocarlo definitivamente. Selection Sort, però, per trovare il prossimo estremo scandisce il suffisso in tempo lineare; Heapsort aveva preparato un heap e trova l'estremo in tempo costante, spendendo soltanto $O(\log n)$ per riparare la struttura dopo ogni estrazione. La lezione generale è più ampia dell'ordinamento: *una struttura dati appropriata può migliorare un algoritmo senza cambiarne l'idea di fondo*.

La storia. J. W. J. Williams pubblicò nel giugno del 1964 l'*Algorithm 232: Heapsort*. Nel dicembre dello stesso anno Robert W. Floyd presentò una versione particolarmente efficiente, associata alla costruzione e gestione in-place dello heap [7, 16]. Siamo così arrivati, storicamente e concettualmente, a una famiglia di algoritmi che raggiunge il tempo $n \log n$ anche nel caso peggiore senza richiedere un array ausiliario lineare.

5 Oltre i classici: cinque idee da esplorare

I metodi studiati finora costituiscono una piccola grammatica di idee: inserire, selezionare, fondere, partizionare, mantenere uno heap. La ricerca successiva non si è limitata a inventare algoritmi completamente nuovi; molto spesso ha riconsiderato questi ingredienti, combinandoli o adattandoli al costo effettivo delle operazioni su un elaboratore.

Le cinque schede che seguono non sostituiscono lo studio dei lavori originali. Ognuna presenta il problema motivante, la principale intuizione e un riferimento verificabile da cui partire. Sono state scelte per mostrare che un algoritmo "classico" non è necessariamente il punto finale di una linea di ricerca.

5.1 Shellsort: inserire senza essere prigionieri dell'adiacenza

Insertion Sort è rapido quando un elemento è già vicino alla sua posizione finale. Nel caso pessimo, però, un elemento molto piccolo collocato alla fine dell'array deve attraversare quasi tutte le celle precedenti. Il problema non è la regola di inserimento: è il fatto che un elemento si possa muovere soltanto di una posizione alla volta.

Nel 1959 Donald L. Shell propose una modifica audace [13]. Invece di confrontare subito elementi adiacenti, scegliamo una distanza g e ordiniamo per inserimento le sottosequenze formate da posizioni che differiscono di g. Successivamente diminuiamo g, ripetendo il lavoro, fino a raggiungere $g = 1$. L'ultimo passaggio è un normale Insertion Sort, ma riceve un array in cui gli elementi hanno già compiuto spostamenti anche molto lunghi.

Per esempio, con $g = 3$ possiamo lavorare sulle posizioni $0, 3, 6, \ldots$, poi $1, 4, 7, \ldots$ e $2, 5, 8, \ldots$. Gli elementi non vengono riuniti in nuovi array: ciascuna sottosequenza è individuata dagli indici. La scelta delle distanze, detta *gap sequence*, è il vero parametro progettuale dell'algoritmo.

La complessità di Shellsort non ammette un unico valore indipendente dai gap. Una sequenza ingenua può lasciare un costo quadratico nel caso peggiore, mentre sequenze ben studiate ottengono limiti migliori. Shellsort è generalmente in-place, ma non stabile. Il suo interesse va oltre le stime asintotiche: mostra come un cambiamento nel *raggio dei movimenti* possa trasformare profondamente il comportamento pratico di una strategia altrimenti elementare.

Per approfondire. D. L. Shell, *A High-Speed Sorting Procedure*, *Communications of the ACM*, 2(7), 30–32, 1959, DOI 10.1145/368370.368387 [13]. La domanda di ricerca naturale è: quanto può migliorare il caso pessimo cambiando soltanto la sequenza dei gap?

5.2 Introsort: accorgersi quando una buona strategia sta fallendo

Quicksort è spesso rapido, ma ha una debolezza ben nota: una lunga successione di partizioni squilibrate può portarlo al costo quadratico. Heapsort ha il problema opposto: garantisce $O(n \log n)$ nel caso peggiore, ma spesso è meno favorevole nell'uso della memoria cache. Perché scegliere una delle due qualità rinunciando all'altra?

L'idea di *Introsort*, proposta da David R. Musser nel 1997, è una forma di autocontrollo algoritmico [11]. L'esecuzione comincia come Quicksort; contemporaneamente si tiene traccia della profondità di ricorsione. Se tale profondità supera una soglia proporzionale a $\log n$, l'algoritmo interpreta il fenomeno come un segnale che le partizioni sono state troppo sbilanciate e passa, per il sottoproblema rimasto, a Heapsort.

Il principio non richiede di prevedere quali input saranno sfavorevoli: il programma osserva *come sta procedendo* e, quando necessario, cambia tattica. Molte implementazioni utilizzano anche Insertion Sort per gli intervalli molto piccoli, dove i costi costanti delle chiamate e delle partizioni possono diventare predominanti.

Nel caso favorevole si conserva il comportamento tipico di Quicksort; nel caso peggiore il numero di livelli di partizionamento prima della sostituzione è limitato, così il lavoro complessivo rimane $O(n \log n)$. Introsort è, in generale, non stabile. Più ancora della specifica combinazione di metodi, conta la sua lezione progettuale: *un algoritmo può misurare un indicatore della propria esecuzione e adattarsi per mantenere una garanzia teorica*.

Per approfondire. D. R. Musser, *Introspective Sorting and Selection Algorithms*, *Software: Practice and Experience*, 27(8), 983–993, 1997 [11]. Si rifletta sulla scelta dell'indicatore: perché una profondità eccessiva è sufficiente a riconoscere un rischio, mentre il valore del pivot da solo non lo è?

5.3 QuickHeapsort: dalla partizione allo heap

Quicksort e Heapsort potrebbero sembrare rivali: il primo partiziona intorno a un pivot, il secondo mantiene un ordine parziale in un albero. Nel 2000 Domenico Cantone e Gianluca Cincotti mostrarono che le due idee possono essere integrate in un unico schema, chiamato *QuickHeapsort* [1, 2].

L'intuizione è riutilizzare la partizione di Quicksort non per ordinare *ricorsivamente entrambe* le parti, ma per trattarne una con una variante di Heapsort. Lo spazio occupato dall'altra parte può essere sfruttato dall'organizzazione del procedimento. Dopo questa fase, il problema di ordinamento rimane soltanto sulla porzione non ancora sistemata. In questo modo la scelta del pivot e la struttura heap non lavorano l'una contro l'altra, ma costituiscono due strumenti complementari.

La descrizione è volutamente concettuale: la variante di heap impiegata nel lavoro originale non coincide semplicemente con la sequenza di istruzioni di Heapsort riportata nella sezione precedente. La gestione degli elementi già estratti e dello spazio disponibile è parte sostanziale del progetto, e deve essere studiata nell'articolo per una ricostruzione completa.

Il lavoro è interessante perché inaugura in modo esplicito una linea di ibridazione nella quale il passaggio di partizionamento viene combinato con un algoritmo ausiliario. La successiva letteratura su *QuickXsort* ha permesso di collocare QuickHeapsort e QuickMergesort entro un quadro comune e di analizzarne con precisione i confronti [5, 15].

Per approfondire. D. Cantone e G. Cincotti, *QuickHeapsort, an Efficient Mix of Classical Sorting Algorithms*, presentato al *CIAC 2000*, pp. 150–162, e pubblicato in *Theoretical Computer Science*, 285(1), 25–42, 2002, DOI 10.1016/S0304-3975(01)00288-2 [2]. Una lettura utile è quella della partizione vista non come soluzione completa, ma come modo per ottenere *risorse* da riutilizzare nel sottoproblema.

5.4 Fast-Insertion-Sort: inserire blocchi anziché elementi

Abbiamo visto che Insertion Sort costruisce una porzione ordinata inserendo un elemento per volta. Perché non provare a inserire gruppi già ordinati di elementi, invece di una singola chiave? È da questa generalizzazione che nasce *Fast-Insertion-Sort*, una famiglia di algoritmi proposta da Simone Faro, Francesco Pio Marino e Stefano Scafiti nel 2020 [6].

L'idea fondamentale è dividere l'array in *blocchi*, ordinarli e inserirli progressivamente nella parte già ordinata attraverso una procedura di inserimento di blocco. Questo permette di sfruttare la struttura delle sequenze in modo più efficace rispetto all'inserimento isolato di ogni elemento. Il meccanismo può poi essere ripetuto su più livelli: anche il metodo utilizzato per ordinare un blocco può essere una versione della stessa strategia.

Nel lavoro originale gli autori analizzano una famiglia annidata, parametrizzata da un intero positivo h, con complessità nel caso peggiore $O(n^{1+1/h})$ per i livelli considerati. Presentano inoltre un'ulteriore generalizzazione ricorsiva con complessità $O(n \log n)$ nel caso peggiore. Le versioni esterne proposte preservano stabilità e adattività; il lavoro discute anche trasformazioni per ottenere versioni in-place a un costo moltiplicativo costante, senza pretendere che tutte le varianti abbiano le medesime proprietà di memoria.

Questa linea di ricerca è didatticamente preziosa perché parte da un algoritmo studiato nelle prime pagine della dispensa, ne conserva la filosofia e mostra come il *livello di granularità* del lavoro possa cambiare l'ordine di grandezza del costo. L'inserimento non è più necessariamente un'operazione elementare su un solo record: può diventare un'operazione su sottosequenze ordinate.

Per approfondire. S. Faro, F. P. Marino e S. Scafiti, *Fast-Insertion-Sort: a New Family of Efficient Variants of the Insertion-Sort Algorithm*, *SOFSEM 2020 Doctoral Student Research Forum*, CEUR Workshop Proceedings 2568, 37–48, 2020. Il testo originale è disponibile a questo indirizzo [6].

5.5 QuickMergesort: il buffer era già nell'array

La forza di Merge Sort è il limite $O(n \log n)$ nel caso peggiore; il suo inconveniente, nella forma elementare, è l'uso di un array di supporto di dimensione lineare. Quicksort, al contrario, suddivide i dati senza allocare un secondo array. Da qui una domanda molto concreta: possiamo ottenere parte dei vantaggi della fusione usando spazio che si trova già *dentro* l'array?

QuickMergesort risponde combinando una partizione alla Quicksort con una fase di Merge Sort. Dopo aver separato l'input rispetto a un pivot, si ordina una delle due parti mediante una variante di Merge Sort che utilizza l'altra come area temporanea. Il contenuto della parte usata come buffer non va perso: mediante scambi, i suoi elementi vengono temporaneamente riorganizzati. Terminata la prima fase, si ordina ricorsivamente la parte rimasta, che continua a contenere gli stessi record, anche se eventualmente in un ordine diverso.

Per realizzare l'idea è indispensabile rispettare un vincolo di capienza: la porzione usata come buffer deve essere sufficiente per la variante di fusione scelta. Non basta dunque sostituire una chiamata di Merge Sort all'interno di Quicksort: la gestione dei segmenti e degli scambi è il cuore dell'algoritmo.

Stefan Edelkamp e Armin Weiß hanno studiato versioni pratiche ed efficienti di QuickMergesort, anche con scelta del pivot progettata per garantire un costo $O(n \log n)$ nel caso peggiore [4]. In un lavoro successivo con Sebastian Wild, QuickMergesort è stato inquadrato nel paradigma più generale *QuickXsort*, nel quale l'algoritmo ausiliario può essere diverso da Merge Sort [5].

Per approfondire. S. Edelkamp e A. Weiß, *Worst-Case Efficient Sorting with QuickMergesort*, *ALENEX 2019*, 1–14, DOI 10.1137/1.9781611975499.1 [4]; S. Edelkamp, A. Weiß e S. Wild, *QuickXsort: A Fast Sorting Scheme in Theory and Practice*, *Algorithmica* 82, 509–588, 2020 [5]. La domanda più fertile è forse questa: quante altre volte una risorsa "mancante" può essere ricavata da parti dell'input temporaneamente inutilizzate?

6 Possiamo fare meglio di $n \log n$ usando soltanto confronti?

Abbiamo incontrato più volte lo stesso ordine di grandezza, $n \log n$. Merge Sort e Heapsort lo raggiungono nel caso peggiore; Quicksort lo raggiunge in media sotto ipotesi appropriate. È solo una coincidenza? O esiste una ragione per cui tanti algoritmi diversi sembrano incontrare lo stesso ostacolo?

La risposta richiede un cambiamento di prospettiva. Finora abbiamo studiato *singoli algoritmi*, analizzando le istruzioni eseguite. Adesso vogliamo dimostrare qualcosa che valga per *qualunque algoritmo* che risolva il problema usando esclusivamente confronti fra chiavi. Per farlo, non descriveremo un programma: descriveremo tutte le informazioni che un programma può acquisire con i confronti.

6.1 Ogni confronto è una domanda con due risposte

Supponiamo che tutte le n chiavi siano distinte. Quando confrontiamo due elementi x e y, possiamo ottenere soltanto una delle due risposte $x < y$ oppure $x > y$. Un algoritmo può usare l'esito per decidere quali elementi confrontare dopo, ma ogni scelta si riduce alla stessa struttura: una domanda con due possibili continuazioni.

Possiamo quindi rappresentare l'esecuzione mediante un *albero di decisione binario*. Ogni nodo interno è un confronto; i suoi due archi rappresentano i possibili esiti; ogni foglia corrisponde a una conclusione del procedimento, cioè a una determinata permutazione delle posizioni di input che produce l'output ordinato.

Per tre elementi distinti, ci sono sei possibili ordini relativi. L'albero che segue mostra un modo di distinguerli, dove "sì" indica che la disuguaglianza scritta nel nodo è vera.

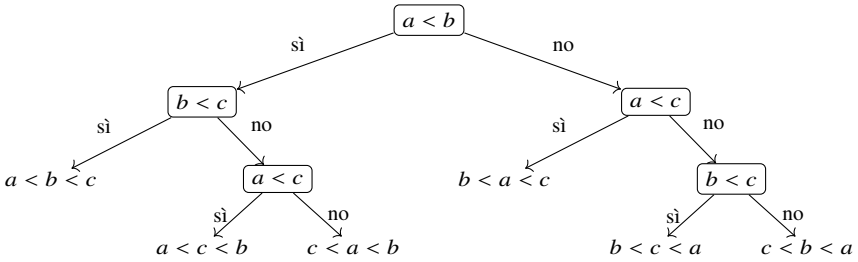


Figura 2: Un albero di decisione per ordinare tre chiavi distinte. Esistono sei risultati possibili e sono necessari fino a tre confronti.

L'osservazione decisiva è che *ordini relativi diversi devono poter essere distinti*. Se due permutazioni dell'input conducessero alla stessa foglia, l'algoritmo prenderebbe per entrambe la stessa decisione finale sulle posizioni dei record: almeno una delle due non potrebbe risultare ordinata correttamente.

6.2 La dimostrazione del limite inferiore

Teorema 6.1 (Limite inferiore dell'ordinamento per confronti)**.** *Ogni algoritmo deterministico corretto che ordina n elementi con chiavi distinte usando soltanto confronti tra chiavi richiede* $\Omega(n \log n)$ *confronti nel caso peggiore.*

Dimostrazione. Le n chiavi distinte possono presentarsi in $n!$ ordini relativi. Pertanto un albero di decisione che descrive un algoritmo corretto deve possedere almeno $n!$ foglie. Un albero binario di altezza h possiede al più 2^h foglie. Se h è il numero massimo di confronti eseguiti lungo una qualunque esecuzione, segue che

$$2^h \geq n!, \qquad \text{e quindi} \qquad h \geq \log_2(n!).$$

Resta da stimare il fattoriale. Per $n \geq 4$, almeno $\lfloor n/2 \rfloor$ fattori nel prodotto $n! = 1 \cdot 2 \cdots n$ sono non inferiori a $n/2$. Di conseguenza,

$$\log_2(n!) \geq \lfloor n/2 \rfloor \log_2(n/2) = \Omega(n \log n).$$

L'altezza dell'albero, cioè il numero di confronti nel caso peggiore, non può quindi essere asintoticamente inferiore a $n \log n$. □

La prova non considera l'implementazione, la presenza di ricorsione, la struttura dell'array, l'uso di uno heap o la strategia con cui si scelgono i confronti. Si applica a tutti gli algoritmi che estraggono informazioni sull'ordine delle chiavi soltanto dai confronti. Per questo è molto più potente di un'analisi di complessità riferita a uno specifico programma.

6.3 Che cosa significa "ottimo"?

Un algoritmo che richiede $O(n \log n)$ confronti *nel caso peggiore* è asintoticamente ottimo nel modello a confronti, perché nessun algoritmo corretto può avere un limite peggiore asintoticamente più piccolo su tutti gli input. Merge Sort e Heapsort raggiungono questa soglia. Quicksort, nella versione elementare studiata, no: esistono input sui quali il costo diventa quadratico. Una variante di Quicksort con garanzia nel caso pessimo, come Introsort, può invece essere ottima nello stesso senso.

Questo risultato non dice che ogni esecuzione debba usare $n \log n$ confronti. Insertion Sort, per esempio, richiede soltanto tempo lineare su un array già ordinato. Dice che *non si può garantire* un numero sub-$n \log n$ di confronti su ogni possibile ordinamento di n chiavi distinte, rimanendo nel modello considerato.

C'è infine un'altra precisazione: la dimostrazione impone un limite al *numero di confronti*, non al tempo richiesto per leggere o scrivere dati in ogni possibile modello di calcolo. Se consentiamo di ricavare informazioni dalla rappresentazione numerica delle chiavi senza confrontarle a coppie, stiamo cambiando modello. Ed è proprio ciò che faremo nelle ultime due sezioni.

7 Oltre i confronti: usare la struttura delle chiavi

Per oltrepassare una barriera matematica non bisogna aggirare una dimostrazione corretta: bisogna individuare quale ipotesi della dimostrazione non è più valida. Nell'albero di decisione ogni confronto risponde a una domanda binaria sull'ordine relativo di due chiavi. Ma se sappiamo che le chiavi sono interi piccoli, perché porre sempre domande del tipo "$x < y$?"? Possiamo usare direttamente il valore della chiave come indirizzo di una cella e raccogliere informazioni con un conteggio.

Questa non è una scorciatoia gratuita. Gli algoritmi che seguono richiedono ipotesi aggiuntive sul dominio delle chiavi e, spesso, ulteriore memoria. In compenso, quando le ipotesi sono soddisfatte, possono raggiungere tempi lineari nella dimensione dell'input.

7.1 Counting Sort: contare invece di confrontare

Supponiamo che ogni chiave sia un intero nell'intervallo $\{0, 1, \ldots, k-1\}$. Se k non è troppo grande, possiamo creare un array $C[0..k-1]$ che memorizzi quante volte compare ogni valore. Nell'input $[4, 2, 2, 0, 3, 2]$ otteniamo

$$C = [1, 0, 3, 1, 1].$$

Leggendo da sinistra a destra questo vettore, sapremmo già quali valori devono comparire nell'output. Se però i valori sono chiavi di record più ricchi, dobbiamo ricostruire l'array ordinato senza perdere i dati associati. Vogliamo inoltre preservare la stabilità.

Il trucco consiste nel trasformare i conteggi in *conteggi cumulativi*. Dopo il passaggio

$$C[v] \leftarrow C[0] + C[1] + \cdots + C[v],$$

$C[v]$ indica quanti elementi hanno chiave al più v. È quindi la posizione, contata da 1, dell'ultima occorrenza di v nell'output. Per l'esempio precedente, il vettore cumulativo è $[1, 1, 4, 5, 6]$. Nella procedura riportata, k viene ricavato dalla massima chiave presente; il risultato viene poi ricopiato in A, in modo che anche l'interfaccia di Counting Sort resti CountingSort(A, n).

Algoritmo 13 Counting Sort stabile su record con chiavi intere non negative.

```
1:  procedura CountingSort(A, n)
2:      se n = 0 allora
3:          restituisci
4:      fine se
5:      k ← 1 + max_{0≤i<n} chiave(A[i])
6:      alloca C[0..k − 1], inizializzato a 0
7:      alloca B[0..n − 1]
8:      per i ← 0 fino a n − 1 fai
9:          v ← chiave(A[i])
10:         C[v] ← C[v] + 1
11:     fine per
12:     per v ← 1 fino a k − 1 fai
13:         C[v] ← C[v] + C[v − 1]
14:     fine per
15:     per i ← n − 1 fino a 0, decrescendo fai
16:         v ← chiave(A[i])
17:         B[C[v] − 1] ← A[i]
18:         C[v] ← C[v] − 1
19:     fine per
20:     per i ← 0 fino a n − 1 fai
21:         A[i] ← B[i]
22:     fine per
23: fine procedura
```

La terza scansione procede *da destra verso sinistra*. È una scelta fondamentale per la stabilità: se due record a e b hanno la stessa chiave e a precede b nell'input, b verrà collocato per primo nell'ultima posizione libera della sua classe; quando arriveremo ad a, la posizione disponibile sarà immediatamente precedente. L'ordine originario sarà dunque conservato.

Correttezza e costo. Il vettore dei conteggi cumulativi assegna a ciascuna classe di chiavi un blocco di posizioni contigue e ordinato rispetto ai blocchi delle altre classi. La scansione finale colloca ogni record esattamente una volta nella posizione libera del suo blocco; perciò l'output è ordinato, contiene gli stessi record ed è stabile.

Le operazioni effettuate sono tre passaggi di lunghezza n o k, quindi

$$T(n, k) = \Theta(n + k), \qquad S(n, k) = \Theta(n + k).$$

La memoria serve per C e per l'output B. Se $k = O(n)$, il tempo è $\Theta(n)$; se k è enormemente maggiore di n, Counting Sort può essere una pessima scelta. Per chiavi nell'intervallo $[a, b]$ è sufficiente usare come indice $v - a$, con $k = b - a + 1$; non è necessario che l'intervallo cominci da zero.

Una parola sulla complessità. Dire che Counting Sort è lineare *in* n senza menzionare k sarebbe scorretto. La sua prestazione dipende dall'ampiezza del dominio delle chiavi, non soltanto dal numero dei record. È il primo esempio della dispensa in cui il contenuto delle chiavi entra direttamente nella complessità dell'algoritmo.

7.2 Radix Sort: ordinare una cifra alla volta

Counting Sort sfrutta un dominio piccolo. Ma se le chiavi sono grandi interi rappresentati da poche cifre o byte, possiamo lavorare su una cifra per volta. Un numero scritto in base b ha una rappresentazione

$$x = d_0 + d_1 b + d_2 b^2 + \cdots + d_{q-1} b^{q-1}, \qquad 0 \le d_i < b.$$

In *Radix Sort* nella variante *LSD* (*least significant digit*, cifra meno significativa per prima) ordiniamo stabilmente prima per d_0, poi per d_1, e così via, fino alla cifra più significativa. L'uso dell'ordinamento stabile in ogni passaggio è indispensabile. Nel codice la base b viene scelta all'interno della procedura, mentre il numero q di cifre viene determinato dalla massima chiave. Per mettere in evidenza il meccanismo, ogni passaggio è scritto esplicitamente come un Counting Sort stabile sulla cifra corrente.

Algoritmo 14 Radix Sort LSD: ogni cifra viene trattata con Counting Sort stabile.

```
1:  procedura RadixSortLSD(A, n)
2:      se n ≤ 1 allora
3:          restituisci
4:      fine se
5:      scegli una base b ≥ 2 (ad esempio 10 o 256)
6:      M ← max_{0≤i<n} chiave(A[i]); q ← 1
7:      mentre M ≥ b fai
8:          M ← ⌊M/b⌋; q ← q + 1
9:      fine mentre
10:     alloca B[0..n − 1] e C[0..b − 1]
11:     per t ← 0 fino a q − 1 fai
12:         azzera C[0..b − 1]
13:         per i ← 0 fino a n − 1 fai
14:             v ← ⌊chiave(A[i])/b^t⌋ mod b
15:             C[v] ← C[v] + 1
16:         fine per
17:         per v ← 1 fino a b − 1 fai
18:             C[v] ← C[v] + C[v − 1]
19:         fine per
20:         per i ← n − 1 fino a 0, decrescendo fai
21:             v ← ⌊chiave(A[i])/b^t⌋ mod b
22:             B[C[v] − 1] ← A[i]; C[v] ← C[v] − 1
23:         fine per
24:         per i ← 0 fino a n − 1 fai
25:             A[i] ← B[i]
26:         fine per
27:     fine per
28: fine procedura
```

Consideriamo le chiavi a due cifre $[21, 13, 11, 23, 12]$. Ordinandole *stabilmente per le unità* otteniamo $[21, 11, 12, 13, 23]$: prima le chiavi che terminano in 1, poi quella che termina in 2, infine quelle che terminano in 3. Ora ordiniamo stabilmente *per le decine*. I valori con decina 1 risultano $[11, 12, 13]$ e conservano il loro ordine sulle unità; quelli con decina 2 risultano $[21, 23]$. Otteniamo infine

$$[11, 12, 13, 21, 23].$$

Perché funziona. Dopo il primo passaggio, i numeri sono ordinati in base alla cifra meno significativa. Supponiamo che, dopo t passaggi, i record siano ordinati rispetto alle ultime t cifre. Un ordinamento stabile per la cifra successiva dispone i record secondo quella cifra e, fra record in cui essa è uguale, conserva il precedente ordine delle t cifre inferiori. Per induzione, dopo q passaggi i numeri sono ordinati rispetto all'intera rappresentazione.

Quanto costa. Se per ciascuna cifra usiamo Counting Sort con b possibili valori, ogni passaggio richiede $\Theta(n + b)$ tempo; per q cifre, otteniamo

$$T(n, b, q) = \Theta\big(q(n + b)\big), \qquad S(n, b, q) = O(n + b).$$

Lo spazio riguarda un passaggio alla volta, con buffer riutilizzabili. Se $b = O(n)$ e il numero di cifre q è limitato da una costante, il tempo è lineare in n. In generale, però, q dipende dalla lunghezza della

rappresentazione e non può essere ignorato. Chiavi negative, lunghezze variabili e dati non numerici richiedono una codifica e una gestione dei segni coerenti con l'ordine desiderato.

Dal censimento alle cifre. Già nel 1887 Herman Hollerith aveva progettato un sistema per elaborare schede perforate con dispositivi di conteggio e smistamento, poi impiegato per il censimento statunitense del 1890 [14]. La separazione meccanica di record sulla base di simboli codificati è un importante antecedente storico degli ordinamenti per distribuzione. Non è necessario attribuire a quelle macchine l'implementazione esatta del moderno Radix Sort LSD: ciò che unisce le due storie è l'intuizione di *sfruttare la rappresentazione dei dati* anziché confrontare coppie di chiavi.

7.3 Perché non abbiamo violato il limite inferiore

La dimostrazione con l'albero di decisione assume che l'algoritmo acquisisca informazioni sull'ordine soltanto attraverso confronti binari. Counting Sort utilizza una chiave come indice di memoria; Radix Sort ne estrae le cifre e ordina in base a esse. Queste operazioni forniscono informazioni sulle chiavi che il modello a confronti non rende disponibili. I due algoritmi non confutano il limite $\Omega(n \log n)$: operano al di fuori delle sue ipotesi.

La distinzione è una delle idee centrali dell'analisi degli algoritmi. Chiedere "qual è l'algoritmo di ordinamento più veloce?" senza specificare quali dati possediamo, quale memoria possiamo usare e quali operazioni hanno costo costante non ammette una risposta universale.

8 Confronto finale e criteri di scelta

Siamo partiti da un array non ordinato, una manciata di cicli e l'idea di mettere ogni elemento al posto giusto. Abbiamo poi scoperto che il modo in cui si mantiene una proprietà durante il calcolo — un prefisso definitivo, un prefisso ordinato, due metà ordinate, una partizione, uno heap — è ciò che determina la forma e spesso la complessità di un algoritmo. Infine, abbiamo visto che perfino un limite inferiore deve essere interpretato alla luce del modello di calcolo.

Tabella 2: Quadro orientativo degli algoritmi approfonditi nel testo. "Atteso" per Quicksort presuppone chiavi distinte e scelta uniforme casuale del pivot.

Algoritmo	Caso migliore	Caso peggiore	Spazio extra	Stabile
Selection Sort (iter.)	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	No
Insertion Sort (iter.)	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	Sì
Bubble Sort (ott.)	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	Sì
Selection Sort ric.	$\Theta(n^2)$	$\Theta(n^2)$	$O(n)$	No
Insertion Sort ric.	$\Theta(n)$	$\Theta(n^2)$	$O(n)$	Sì
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Sì
Quicksort (proposto)	$\Theta(n \log n)$	$\Theta(n^2)$	$O(n)$ peggiore	No
Heapsort	$\Theta(n)$	$\Theta(n \log n)$	$O(1)$	No
Counting Sort	$\Theta(n + k)$	$\Theta(n + k)$	$O(n + k)$	Sì
Radix Sort LSD	$\Theta(q(n + b))$	$\Theta(q(n + b))$	$O(n + b)$	Sì

La tabella sintetizza il costo, ma non basta a scegliere. Per piccoli input quasi ordinati, Insertion Sort può essere un'opzione eccellente. Se sono necessarie stabilità e una garanzia temporale sul caso peggiore, Merge Sort è una scelta naturale quando la memoria ausiliaria non è un problema. Se vogliamo un limite peggiore di $O(n \log n)$ con memoria costante, Heapsort offre una risposta chiara. Se la distribuzione dei dati è favorevole e il costo delle operazioni elementari conta molto, varianti ben progettate di Quicksort

sono spesso competitive. Se le chiavi appartengono a un dominio numerico limitato, vale la pena abbandonare il modello a confronti.

8.1 Un'ultima prospettiva

L'ordinamento è spesso presentato come una collezione di nomi da memorizzare. Questa dispensa ha cercato di mostrarlo piuttosto come una successione di *domande progettuali*. Posso evitare confronti ripetuti? Posso mantenere un'invariante più forte? Posso dividere meglio il problema? Posso introdurre una struttura dati che renda economica l'operazione cruciale? Posso sfruttare informazioni sulle chiavi che un confronto non rivela?

Sono domande che ricompaiono molto lontano dall'ordinamento: negli algoritmi su grafi, nell'indicizzazione, nella gestione delle code di priorità, nella ricerca di stringhe e nei problemi di ottimizzazione. È qui che lo studio degli algoritmi di ordinamento raggiunge il suo vero obiettivo: insegnare un metodo per *progettare e analizzare algoritmi*, non soltanto una tecnica per riordinare array.

Riferimenti bibliografici

[1] Domenico Cantone and Gianluca Cincotti. QuickHeapsort, an efficient mix of classical sorting algorithms. In *Algorithms and Complexity (CIAC 2000)*, pages 150–162. Springer, 2000. doi: 10.1007/3-540-46521-9_13.

[2] Domenico Cantone and Gianluca Cincotti. QuickHeapsort, an efficient mix of classical sorting algorithms. *Theoretical Computer Science*, 285(1):25–42, 2002. doi: 10.1016/S0304-3975(01)00288-2.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.

[4] Stefan Edelkamp and Armin Weiß. Worst-case efficient sorting with QuickMergesort. In *Proceedings of the Twenty-First Workshop on Algorithm Engineering and Experiments (ALENEX 2019)*, pages 1–14. SIAM, 2019. doi: 10.1137/1.9781611975499.1.

[5] Stefan Edelkamp, Armin Weiß, and Sebastian Wild. QuickXsort: A fast sorting scheme in theory and practice. *Algorithmica*, 82(3):509–588, 2020. doi: 10.1007/s00453-019-00634-0.

[6] Simone Faro, Francesco Pio Marino, and Stefano Scafiti. Fast-Insertion-Sort: a new family of efficient variants of the insertion-sort algorithm. In *Proceedings of the SOFSEM 2020 Doctoral Student Research Forum*, volume 2568 of *CEUR Workshop Proceedings*, pages 37–48, 2020. URL https://www.dmi.unict.it/faro/papers/conference/faro55.pdf.

[7] Robert W. Floyd. Algorithm 245: Treesort 3. *Communications of the ACM*, 7(12):701, 1964.

[8] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962. doi: 10.1093/comjnl/5.1.10.

[9] Donald E. Knuth. Von neumann's first computer program. *ACM Computing Surveys*, 2(4):247–260, 1970. doi: 10.1145/356580.356581.

[10] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2 edition, 1998. ISBN 9780201896855. URL https://cs.stanford.edu/~knuth/taocp.html.

[11] David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/(SICI)1097-024X(199708)27:8%3C983::AID-SPE117%3E3.0.CO;2-%23.

[12] Mark Priestley. *Routines of Substitution: John von Neumann's Work on Software Development, 1945–1948*. Springer, 2018. doi: 10.1007/978-3-319-91671-2.

[13] Donald L. Shell. A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32, 1959. doi: 10.1145/368370.368387.

[14] Smithsonian Institution. From herman hollerith to ibm. National Museum of American History, sezione Tabulating Equipment, s.d. URL https://www.si.edu/spotlight/tabulating-equipment/from-herman-hollerith-to-ibm. Consultato nel 2026.

[15] Sebastian Wild. Average cost of QuickXsort with pivot sampling. In *29th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms (AofA 2018)*, volume 110 of *Leibniz International Proceedings in Informatics*, pages 36:1–36:19. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.AofA.2018.36.

[16] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6):347–348, 1964. doi: 10.1145/512274.512284.